

Overview

The objective of this project was to design and implement a control solution for autonomously controlled TurtleBot3 Waffle Pi robots to collect contaminated (red) and clean (blue) barrels distributed in clusters across a walled arena and deposit them in green collection zones, while minimising the contamination a robot accrues while towing the red barrels.

My methodology combines a deliberative global layer – the ROS 2 Nav2 stack – with a reactive behavioural layer implemented as a finite state machine (FSM) in each robot's controller.

This hybrid was chosen because the task decomposes naturally into discrete phases (search, approach, dock, transport, deposit, decontaminate), each with distinct sensing and actuation needs, whereas precise barrel docking and obstacle avoidance demand tight closed-loop sensor feedback that a planner alone does not provide.

Perception reuses the provided colour-blob visual sensor; I fuse blob area with the LiDAR range along the barrel's bearing to estimate distance and convert detections into world-frame sightings. Coordination is fully decentralised: robots publish target claims and sightings on shared topics so that up to three robots divide the work without a central planner, and the system degrades gracefully to a built-in waypoint navigator when Nav2 is unavailable.

Architecture

This section discusses the system's architecture, which follows one organising principle: everything specific to a single robot is replicated per robot and namespaced, while everything the team must agree on travels over a small number of shared topics.

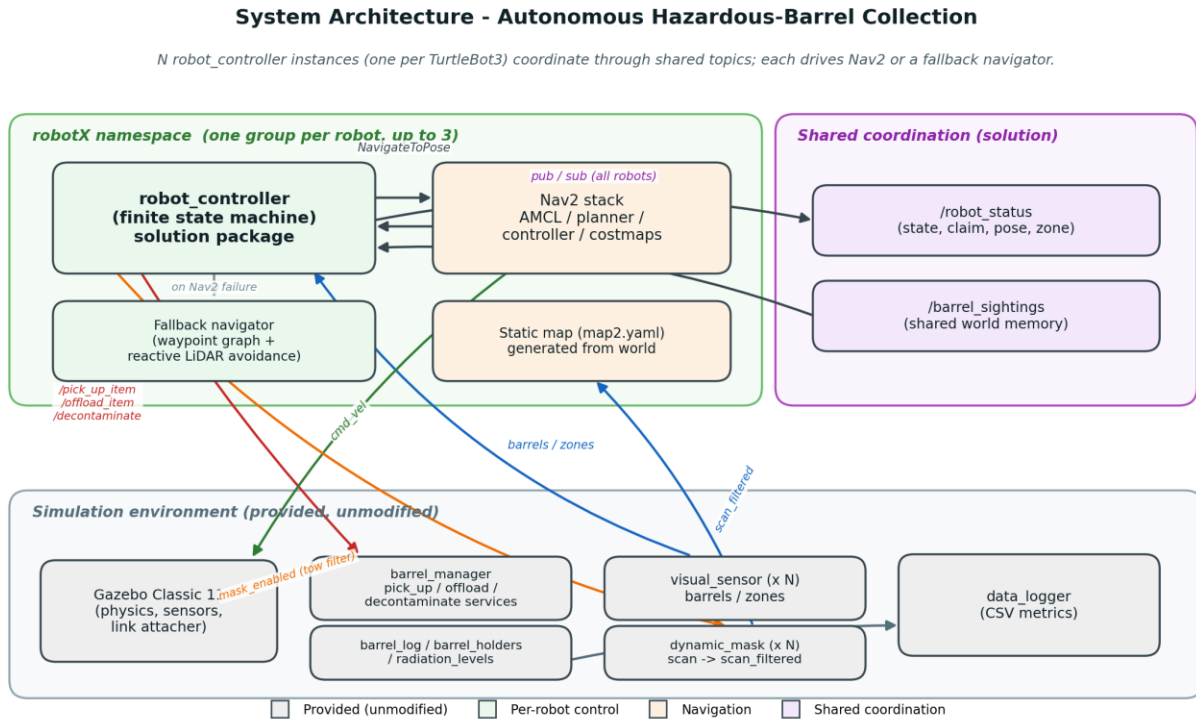


Figure 1. High-Level Architecture.

Green: per-robot control (solution); **orange:** navigation; **purple:** shared coordination topics; **grey:** the provided, unmodified simulation environment.

The system runs one control group per robot, namespaced robotX (*Figure 1*). At its centre is the robot_controller node (the solution package), a 10 Hz finite state machine that orchestrates the mission. It consumes barrels and zones from the provided visual_sensor, and the /barrel_holders and /radiation_levels telemetry from the barrel_manager. It actuates through three provided services – /pick_up_item, /offload_item and /decontaminate – and through velocity commands.

Long-range motion is delegated to the Nav2 stack via the NavigateToPose action. AMCL localises the robot on a static map (map2.yaml) that I rasterise offline from the world's collision meshes, so the map frame coincides with the Gazebo world frame and AMCL poses can be compared directly against the known zone coordinates. When Nav2 repeatedly fails (typically multi-robot corridor congestion) or is disabled, the controller falls back to an internal waypoint-graph navigator with reactive LiDAR avoidance.

Multi-robot coordination is achieved through two custom topics in the `solution_interfaces` package: `/barrel_sightings`, a shared world memory of detected barrels, and `/robot_status`, on which each robot broadcasts its state, pose, claimed target and destination zone. There is no central node; each controller independently avoids claimed targets and congested zones, so the design scales from one to three robots by launch parameter alone. A provided `dynamic_mask` node is toggled to hide a towed barrel from the LiDAR used by Nav2.

Control

Control is arbitrated by one finite state machine per robot, which suits a mission that decomposes into phases with different sensing and actuation regimes rather than one continuous control law.

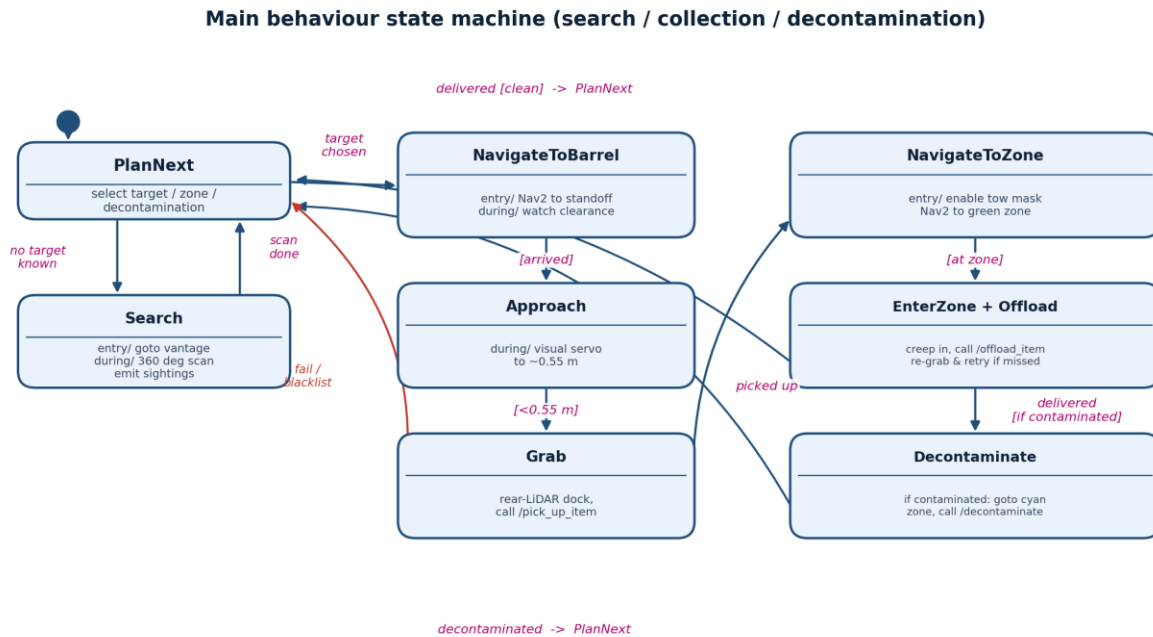


Figure 2. Main behaviour state machine.

Search

When no unclaimed barrel is known, PlanNext sends the robot to the next vantage point covering the barrel clusters; Search performs a 360° scan, converting camera detections into world-frame sightings that are shared with the team.

Collection

Once a target is chosen, NavigateToBarrel drives Nav2 to a 0.9 m standoff; Approach visually servos onto the barrel to ~0.55 m, fusing blob area with the LiDAR ray for range; Grab executes a rear-LiDAR docking sequence (turn 180°, centre the barrel behind the robot, reverse) before calling /pick_up_item.

Deposit

While towing, the LiDAR tow-mask is enabled and NavigateToZone heads for the robot's allocated green zone; EnterZone + Offload creeps in and calls /offload_item, re-grabbing and retrying on a different lane if the drop misses the boundary.

Decontamination

After each delivery a contaminated robot detours to the cyan zone, which lies on the return route, and calls /decontaminate.

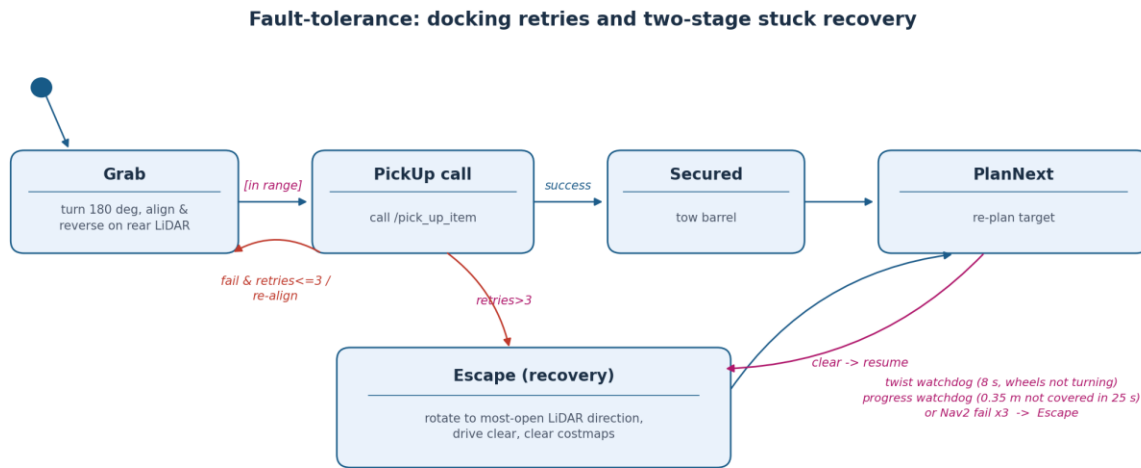


Figure 3. Fault-tolerance state machine: bounded docking retries and two-stage stuck recovery.

Fault tolerance addresses the required fault classes.

Failure to collect: a failed /pick_up_item triggers up to three re-align-and-retry cycles, after which the barrel is black-listed and re-planned. Failure to avoid obstacles / getting stuck: two watchdogs act. One detects 8 s without wheel motion; because rotating on the spot defeats it, a second requires 0.35 m of real displacement every 25 s, catching avoidance oscillation. Either, or three Nav2 failures, invokes Escape, which rotates toward the most open LiDAR direction and drives clear.

Missed drops, rejected goals during Nav2 start-up, and stale localisation are each handled by dedicated guarded transitions.

Evaluation

This section outlines my evaluation of the solution using three complementary methodologies: experimental simulation, data analysis of logged missions, and complexity analysis.

1. Experimental Simulation

Each of the five scenarios was run three times – 15 missions of 840 s, each in a fresh container with a fresh build, at real-time factor 0.85–1.00. The outcome is bimodal: 8 of the 15 delivered nothing, the others one to three barrels.

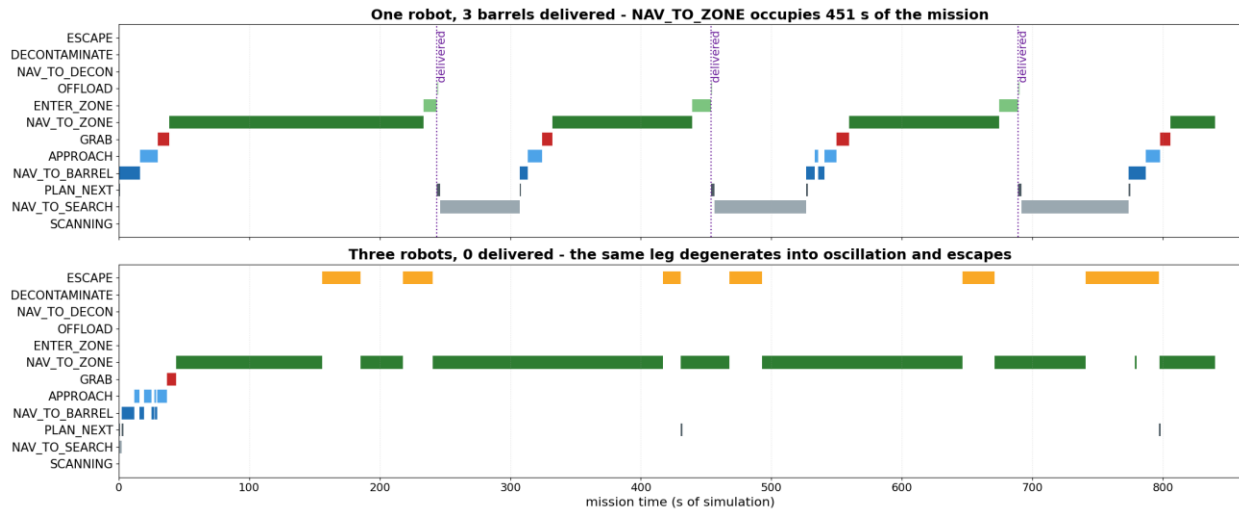


Figure 4. Top: one robot delivering three barrels, with NAV_TO_ZONE occupying 451 s of the mission. **Bottom:** robot 1 of a three-robot mission that delivered nothing, its transport leg broken up by repeated escapes.

2. Data Analysis

Instrumentation turned a persistent failure into two measurable root causes.

Early builds collected barrels but rarely delivered them; Gazebo ground truth against the AMCL estimate revealed a 4.6 m localisation error only while towing, because the dynamic_mask's blanked rear sector reads as free space that contradicts the map and diverges the particle filter. The raw scan cut the error to 0.3 m.

Secondly, nine consecutive failed to create plan events followed each dock: reversed into a 10-barrel cluster, the robot is enclosed by lethal costmap cells and Nav2 cannot plan at all. Leaving on the costmap-independent waypoint route fixed this.

Across the 15 missions, 33 pick-ups produced 11 zone arrivals and 11 deliveries (*Figure 5*). Every robot that reached a zone offloaded successfully and no drop missed the boundary, so transport converts 33% while the offload stage is perfect.



Figure 5. Results over 15 missions.

Left: deliveries per run by scenario, three repeats each.

Centre: the pick-up to delivery funnel.

Right: AMCL position error while towing, against Gazebo ground truth, for the two scan sources.

3. Complexity Analysis

Per tick the dominant costs are target selection, $O(S \cdot R)$ for S sightings and R robots, and the fallback planner's Dijkstra search, $O(E \log V)$ on a fixed 25-node graph – both are negligible at 10 Hz.

Throughput is bounded instead by travel time: clusters lie ~15 m from the zones at a 0.24 m/s limit. Success then tracks oscillation rather than distance: runs with at most one no-progress escape per robot averaged 1.50 deliveries against 0.45 for the rest.

Strengths:

Collection and offload are reliable: 33 docks, and every zone arrival delivered.

The dual-navigator arrangement is what makes delivery possible at all: the deliberative planner is unusable inside a cluster, whereas the reactive route is indifferent to costmap occupancy.

Coordination assigns different barrels and zones per robot, and the corridor reservation was observed arbitrating in both directions without starving either robot.

Weaknesses:

The headline weakness is that throughput neither scales nor repeats: per-scenario means are 2.00, 0.00, 0.33, 0.67 and 0.67 (*Figure 5*), and seed 0 gives scenarios 1-3 and 5 an identical layout that still yielded 0, 3 and 3. Transport, not perception or computation, is the bottleneck. Contamination follows it, averaging 599 and peaking at 2974, because a robot that never reaches a zone never decontaminates.

Appraisal:

A faster team would need the bottleneck removed rather than arbitrated – overlapping routes using both zones concurrently, or pipelining collection against transport. Spatial partitioning was tried and measured worse, so claim-based deconfliction replaced it. No configuration is yet dependable; laden egress from a cluster is the highest-value fix.

Methodological Note:

An earlier iteration blamed these failures on multi-robot congestion. That was wrong: the same failure reproduced with a single robot and no peer present, which prompted the ground-truth comparison above. Single runs likewise overstated throughput; only repeated missions exposed the variance.

Safety

This section outlines the HAZOP (Hazard and Operability Study) methodology, summarised by the table below, that was used to identify potential safety hazards for real-world deployment.

ID	Guiding Words	Deviation	Possible Causes	Consequences	Mitigating Actions
H 1	NO	Barrel not gripped / dropped in transit	Failed attach, collision knocks barrel loose, localisation error	Contaminated item left in the open; spreads contamination	Bounded re-grab retries and drop-detection re-pickup
H 2	MORE	Robot over-contaminated	Towing red barrels too long without decontamination	Unreliable operation; contamination spread to clean areas	Proactive decontamination after each delivery
H 3	REVERSE	Robot reverses into a person or an obstacle while docking	Blind rear motion during the grab sequence	Injury or equipment damage	Rear-LiDAR gating stops reversing at 0.17 m
H 4	AS WELL AS	Robot collides with another robot	Corridor congestion, both mark each other as obstacles	Blocked task, mechanical damage, spill	Costmap obstacle layer and stuck-recovery escape
H 5	LATE	Emergency stop / command not honoured in time	Node crash, control loop stall, comms dropout	Robot continues moving uncontrolled with a hazardous load	Watchdog and graceful Nav2 fallback

H 6	OTHER THAN	Wrong item classified as a barrel	Camera noise, similar-coloured clutter	Robot grabs an unintended / unsafe object	Blob area, aspect-ratio and range gating reject spurious detections
----------------	---------------	---	--	---	--

Table 1. HAZOP-style hazard identification for a real-world deployment.

Safety Requirements

Real-world deployment requires redundant obstacle detection (LiDAR + camera + bumpers), human safety zones with thermal/motion sensors, fail-safe defaults (stop on communication loss), hardware emergency stops independent of software, and dual confirmation before critical actions (pickup/offload).

Safety Integrity Level

Obstacle avoidance requires SIL 2; human detection and emergency stop require SIL 3.

While the simulation-based solution provides a foundation for hazardous materials cleanup, real-world deployment would require significant additional safety engineering to address the identified hazards and meet appropriate safety integrity levels.

Ethics

This section outlines my examination of the ethical implications of deploying autonomous robots for hazardous materials cleanup.

Ethical Benefits

The primary ethical justification is the protection of human workers from dangerous exposure. Robots can operate in high-radiation environments where human exposure would be harmful, avoiding chemical burns, inhalation risks, and biological hazards. This solution models contamination accumulation, demonstrating awareness of exposure risks.

Autonomous systems also provide improved response capability — 24/7 operation without fatigue-related errors — and environmental protection through faster, more consistent cleanup to reduce contamination spread.

Ethical Concerns

Safety & Accountability

A robot that mishandles a hazardous item can spread contamination rather than contain it. Clear accountability for failures, thorough validation, and conservative fail-safe behaviour (halt and call for help rather than improvise) are ethical requirements, not optional extras.

My design leans this way – bounded retries then re-planning rather than forcing risky actions – but a real deployment would need certified e-stops and human oversight.

Labour & Dual Use

Deploying robots displaces the workers who currently do this dangerous job. Responsible adoption should retrain and redeploy staff rather than simply cut roles.

The same autonomy that clears hazardous waste could be repurposed to move dangerous materials for harmful ends, so access and intended use warrant governance.

Environmental Justice

Contamination disproportionately affects poorer communities. The prioritisation logic (here, nearest-first with a contamination penalty) is an ethical choice about whose environment is cleaned first, and should be set transparently and equitably.

Transparency & Trust

Because the system logs every collection and contamination level, its actions are auditable – an ethical asset.

Finally, over-trusting an imperfect system is itself a hazard: operators must understand that a reported clean-up is only as reliable as the perception and localisation beneath it, which noise and drift can degrade.

Simulation Scenarios

This section discusses the five launch configurations (defined in .devcontainer/devcontainer.json) that exercise different aspects of the solution. Together they vary team size, perception quality, odometry source, item distribution, the presence of obstacles, and the navigation stack itself, so that both the primary objective (collection) and the secondary objective (contamination) are validated under progressively harder conditions.

The table below specifies the scenarios, including, for each scenario, the launch parameters that are used and their values.

Parameter	Scenario 1	Scenario 2	Scenario 3	Scenario 4	Scenario 5
Name	Baseline	Team	Noisy	Generality	Degraded nav
<i>num_robots</i>	1	3	2	2	1
<i>use_nav2</i>	true	true	true	true	false
<i>sensor_noise</i>	false	false	true	false	false
<i>odometry</i>	ENCODER	ENCODER	ENCODER	ENCODER	WORLD
<i>other</i>	-	-	-	random_seed=42; obstacles=false; red_penalty=2.0	-
What it validates	Full pipeline end-to-end, no confounds	Decentralised claiming, zone allocation, corridor avoidance	AMCL drift correction and rejection of noisy detections	New item layout, empty room, contamination-avoidance bias	Fallback waypoint navigator; graceful degradation without Nav2

Table 2. The five provided simulation scenarios and their validation purpose.

The progression is deliberate.

- Scenario 1 confirms correctness in isolation.
- Scenario 2 stresses the coordination layer with the maximum three robots.
- Scenario 3 attacks perception and localisation with sensor noise on encoder odometry, so AMCL must correct drift.
- Scenario 4 changes the random seed and removes the room obstacles to prove the controllers are not over-fitted to one layout, and applies a red-barrel penalty to favour the contamination objective.
- Scenario 5 disables Nav2 entirely; the built-in navigator delivered in two of its three runs, no worse than the Nav2 configurations, supporting the fault-tolerance claim under severe degradation.

This systematic approach to scenario selection ensures that both nominal operation and edge cases are thoroughly tested.